

Proyecto 1 – Etapa 2

Yesid Steven Piñeros Piñeros - 202013148

Juan David Guevara Arévalo - 202116875

Esteban Orjuela Perdomo - 202211227

Sección 1: Proceso de automatización del proceso de preparación de datos, construcción del modelo, persistencia del modelo y acceso por medio de API:

El proyecto implementa un sistema completo de clasificación de noticias falsas mediante un pipeline de procesamiento de texto y un clasificador basado en XGBoost, expuesto a través de una API REST desarrollada con Flask.

La API implementada cuenta con dos endpoints principales:

- **Endpoint de predicción** (/api/predict_json y /api/predict_csv): Permite recibir tanto noticias individuales en formato JSON como conjuntos de datos en formato CSV. El procesamiento de los datos se realiza automáticamente mediante el pipeline, que ejecuta secuencialmente la tokenización, preprocesamiento (minúsculas, eliminación de puntuación, caracteres no ASCII y stopwords), lematización, vectorización TF-IDF y finalmente la clasificación. El endpoint devuelve la clasificación binaria (0: noticia verdadera, 1: noticia falsa) junto con las probabilidades asociadas a cada clase, manteniendo la estructura original de los datos enviados.
- **Endpoint de reentrenamiento** (/api/retrain): Recibe un conjunto de datos etiquetados para actualizar el modelo. En lugar de evaluar únicamente los enfoques, se tomó la decisión de implementar los tres enfoques distintos con el objetivo de hacer más evidente la diferencia entre los mismos. Los tres enfoques escogidos fueron: reentrenamiento completo, reentrenamiento incremental y ajuste fino.
- **Reentrenamiento completo:** Combina los datos originales de entrenamiento con los nuevos datos proporcionados y entrena el modelo desde cero con este conjunto ampliado. Esto permite que el nuevo modelo aprenda de forma equilibrada a partir de todos los datos, sin embargo, es computacionalmente intensivo si el volumen de datos es muy grande. Además, si el conjunto de datos a agregar es significativamente más pequeño y muy distinto del original, este enfoque podría no ser el más adecuado ya que el conjunto puede ser interpretado como “outlier”.

- **Reentrenamiento incremental:** Actualiza el modelo existente entrenándolo únicamente con los nuevos datos, sin considerar los datos originales. Esto proporciona mayor eficiencia computacional y hace al modelo más robusto frente a diferencias drásticas entre conjuntos de datos. No obstante, si el nuevo conjunto de datos es muy pequeño, el modelo podría no tener suficiente información para lograr buenos resultados en la tarea de aprendizaje automático, ya que “olvidaría” información relevante de los datos antiguos.
- **Ajuste fino:** Este enfoque utiliza el modelo existente junto con los nuevos datos para ajustar sus parámetros, permitiendo aprovechar y refinar los patrones o características aprendidos previamente. Esto puede mejorar el rendimiento del modelo en un nuevo entorno o conjunto de datos. Sin embargo, si el nuevo conjunto de datos es significativamente diferente, el ajuste fino podría no alcanzar el rendimiento esperado en el nuevo contexto y, al mismo tiempo, perder eficacia en el conjunto original.

Cada uno de los enfoques previamente mencionados ofrece un gran valor dependiendo de los datos a utilizar. Por ejemplo, a partir de la conclusión y el análisis realizados en la etapa 1 del proyecto, se comprendió que el modelo funciona en un contexto de noticias españolas relacionadas con política.

Si el nuevo conjunto de datos corresponde a noticias sobre política en Colombia, un ajuste fino podría ser la mejor opción, ya que permitiría adaptar el modelo a las particularidades del nuevo contexto sin perder el conocimiento adquirido previamente.

Por otro lado, si además de noticias sobre política en Colombia se desea incorporar información sobre economía en distintos países hispanohablantes, un reentrenamiento completo sería más adecuado. Este enfoque entrenaría el modelo desde cero con el conjunto de datos ampliado, asegurando que aprenda de manera equilibrada sobre todos los temas y regiones.

En cambio, si se quiere actualizar el modelo para que analice noticias sobre avances tecnológicos, el contexto es tan distinto del contexto político que lo mejor sería reentrenar desde cero el modelo con el nuevo conjunto de datos, es decir, reentrenamiento incremental.

Sección 2: Desarrollo de la aplicación y justificación.

- Usuario final:

El usuario principal de la aplicación es un analista de información o un profesional encargado de la detección de noticias falsas dentro del ámbito gubernamental, en organizaciones internacionales, o en medios de comunicación. Este individuo tendría la responsabilidad de identificar y mitigar la propagación de noticias falsas que puedan afectar la gobernabilidad, la confianza pública, o las relaciones internacionales. Por ello, la aplicación puede ser utilizada por periodistas, investigadores académicos, o equipos de ciberseguridad que trabajan en la validación de información en tiempo real.

La conexión entre la aplicación y el proceso de negocio radica en su capacidad para automatizar la detección de noticias falsas mediante el uso de modelos de aprendizaje automático. Este proceso se integra en flujos de trabajo donde la verificación de información es crítica como en la publicación de noticias, la toma de decisiones políticas, o la evaluación de riesgos en campañas de desinformación. Por ejemplo, un analista puede utilizar la aplicación para evaluar rápidamente la veracidad de una noticia antes de que sea difundida, reduciendo así el impacto de información errónea en la opinión pública o en decisiones estratégicas.

Por lo anterior, la importancia de esta aplicación para el rol mencionado es significativa ya que proporciona una herramienta eficiente y automatizada para la detección de noticias falsas, lo que permite ahorrar tiempo y recursos en comparación con métodos manuales de verificación.

- Explicación de la aplicación:

La aplicación implementada se realizó en Flask para permite la clasificación de noticias como verdaderas o falsas y el reentrenamiento del modelo de clasificación desde una interfaz web, por ello se da la oportunidad de usarla por la web o mediante una API REST. A continuación, se explica su funcionamiento:

1. Estructura General

La aplicación está organizada en módulos que separan la lógica de negocio, la interfaz web, y las operaciones relacionadas con el modelo de aprendizaje automático. Los componentes principales son:

- `app/back_logic/`: Contiene la lógica de la API y los endpoints de predicción
- `app/models/`: Incluye la definición del modelo, cargadores y utilidades para el procesamiento

- `app/web_api/`: Implementa la interfaz web y los endpoints para la interacción del usuario

Dentro de esta estructura destacan dos archivos fundamentales que contienen los endpoints solicitados:

- `app/back_logic/endpoints.py`: Define los endpoints de la API para predicción y reentrenamiento
- `app/web_api/routes.py`: Implementa los endpoints web que interactúan con la API para ofrecer la interfaz de usuario

2. Interfaz Web

Primer Endpoint Solicitado: Clasificación (`/clasificar`)

El endpoint `/clasificar`, implementado en `routes.py`, funciona con la interfaz principal para el usuario que quiere detectar automáticamente si se está procesando una noticia individual o un lote de noticias en CSV, adaptando su comportamiento en consecuencia.

En el archivo `app/web_api/routes.py`, el endpoint de predicción se implementa a través de la ruta `/clasificar` y puede llegar a manejar dos escenarios:

1. **Clasificación de una noticia individual:**
 - a. Recibe título, descripción y fecha mediante un formulario web
 - b. Crea un objeto JSON y lo envía a la API (`/predict_json`)
 - c. Muestra el resultado con la clasificación y las probabilidades
2. **Clasificación por lotes mediante CSV:**
 - a. Recibe un archivo CSV con múltiples noticias
 - b. Envía el archivo a la API (`/predict_csv`)
 - c. Muestra una tabla con todas las predicciones y sus probabilidades

La respuesta en ambos casos cumple con el requisito de devolver, para cada instancia recibida y en el mismo orden la predicción realizada (0: verdadera, 1: falsa) y la probabilidad asociada a cada clase.

Segundo Endpoint Solicitado: Reentrenamiento (`/retrain`)

El endpoint `/retrain`, implementado al igual en `routes.py`, permite al usuario cargar datos etiquetados, seleccionar una estrategia de reentrenamiento, y obtener métricas de rendimiento, actualizando si automáticamente el archivo binario del modelo.

La implementación web del endpoint de reentrenamiento se encuentra en la ruta `/retrain` y permite:

1. Cargar un archivo CSV con datos de entrenamiento que contienen tanto las características (Descripcion) como la variable objetivo (Label)
2. Seleccionar un tipo de reentrenamiento entre tres opciones disponibles
3. Enviar los datos a la API para realizar el reentrenamiento
4. Mostrar las métricas de desempeño resultantes de Precision, Recall, F1-score, y Accuracy

Como resultado del reentrenamiento, el archivo binario del modelo se reemplaza para ser utilizado en futuras predicciones, cumpliendo con el requisito de actualización del modelo.

3. Recursos Informáticos Requeridos:

Para entrenar, ejecutar, persistir el modelo analítico y desplegar la aplicación se necesitan recursos informáticos que incluyan un servidor con capacidad suficiente para manejar tanto el procesamiento de datos como la ejecución del modelo, requiriendo que el mismo cuente con al menos 8 GB de RAM, un procesador multinúcleo y almacenamiento de al menos 50 GB para guardar los datos de entrenamiento y los modelos generados. Además, se requiere un entorno de desarrollo y ejecución como Python 3.x con bibliotecas específicas (Flask, scikit-learn, pandas, etc.) y acceso a la web ya sea por medio de un servicio en la nube como AWS, Azure, o Google Cloud, o un servidor local con acceso a internet para garantizar la disponibilidad de la aplicación.

4. Integración con la Organización:

La aplicación se integrará a la organización como una herramienta de soporte en los procesos de verificación de información y toma de decisiones. Por ello, si la organización cuenta con un flujo de trabajo relacionado con la validación de noticias o la gestión de riesgos de desinformación, la aplicación puede conectarse directamente a estos procesos mediante su API REST para que otros sistemas internos envíen noticias para su clasificación y reciban las predicciones en tiempo real. Por otro lado, la interfaz web podría estar disponible para analistas o usuarios finales que necesiten interactuar manualmente con la herramienta, por lo que sería pertinente que sea desplegada en un servidor accesible a través de una red interna o pública dependiendo de los requisitos de seguridad de la organización.

5. Riesgos para el Usuario Final:

En primer lugar, existe el riesgo de errores en las predicciones del modelo, ya que ningún modelo es 100% preciso y podría llevar a clasificar incorrectamente una noticia verdadera como falsa o viceversa, afectando la confianza del usuario o generar decisiones basadas en información errónea. En segundo lugar, si los datos utilizados para entrenar el modelo no son representativos o están sesgados, el modelo podría perpetuar esos sesgos, afectando la equidad y la calidad de las predicciones. Además, existe el riesgo de fallos técnicos como interrupciones en el servidor, problemas de conectividad, o pérdida de datos, los cuales podrían impactar la disponibilidad de la aplicación. Por último, si no se implementan medidas de seguridad adecuadas, como cifrado de datos y autenticación de usuarios, la aplicación podría ser vulnerable a ataques cibernéticos y comprometer la privacidad de los datos y la integridad del sistema. Por lo anterior, para mitigar estos riesgos es fundamental realizar pruebas exhaustivas, implementar controles de calidad y garantizar la seguridad del entorno de despliegue.

Sección 3: Resultados

El video puede ser encontrado en la wiki del proyecto. En esta sección se discutirá únicamente el análisis realizado sobre la facilidad de uso con base en dos entrevistas y pruebas simples realizadas a 2 estudiantes de la universidad de los andes, de ingeniería de sistemas. Uno de los estudiantes (Juan José) no ha cursado la materia, el otro (Gabriel) la cursó el semestre pasado.

Durante las pruebas de usabilidad, Juan José destacó la facilidad de uso de las funciones principales de clasificación. Encontró la navegación clara y los formularios para ingresar una noticia individual o cargar un archivo CSV muy intuitivos, apreciando las instrucciones directas y los ejemplos de formato. Resaltó como especialmente útil la presentación de resultados, mencionando que la distinción por colores y las barras de probabilidad le permitían evaluar rápidamente la fiabilidad de una noticia, agilizando su proceso de filtrado inicial. Por su parte, Gabriel, evaluando desde una perspectiva más técnica, consideró la interfaz altamente funcional. Si bien coincidió en la simplicidad de las opciones de clasificación, valoró positivamente la sección de reentrenamiento, calificándola como clara y útil para usuarios expertos. Subrayó la ventaja de poder seleccionar directamente entre las distintas estrategias de reentrenamiento y visualizar inmediatamente las métricas de desempeño resultantes, considerando que las descripciones proporcionadas ayudaban a tomar una decisión informada sobre qué método utilizar.

Dado lo anterior, se concluye que la aplicación es eficiente para usuarios que solo querrían evaluar una noticia, y para usuarios que están interesados en datos u operaciones más técnicas sobre el modelo.

Sección 4: Trabajo en equipo

Líder de proyecto e Ingeniero de Datos: Yesid Piñeros

Puntos: 34/100

Tareas realizadas:

- Coordinación general del equipo para esta etapa del proyecto, definición de reuniones y seguimiento de tareas.
- Responsable de la automatización del preprocesamiento y entrenamiento: Implementación de los transformadores, creación y refinamiento del pipeline completo, entrenamiento inicial del pipeline y asegurar la correcta carga del modelo.
- Configuración inicial de la estructura del proyecto Flask.
- Video con resultados obtenidos.

Tiempo dedicado: 12 horas

Ingeniero de Software (API): Juan David Guevara Arévalo

Puntos: 33/100

Tareas realizadas:

- Implementación del endpoint para predicciones individuales y múltiples.
- Implementación del endpoint para el reentrenamiento.
- Desarrollo de la lógica de reentrenamiento.
- Cálculo de métricas post-reentrenamiento.
- Manejo adecuado del archivo .joblib.

Tiempo dedicado: 12 horas

Ingeniero de Software (Frontend): Esteban Orjuela Perdomo

Puntos: 33/100

Tareas realizadas:

- Diseño de la interfaz de usuario y flujo de navegación.
- Interacción entre la interfaz y la API interna.
- Consolidación final del informe
- Correcto manejo de respuestas de la API.
- Entrevistas de utilidad y facilidad de uso.
- Revisión general de la entrega final.

Tiempo dedicado: 12 horas